

Pre-Processing Modeling Artefacts for Graph Neural Networks: A DSL for Model Encoding

Zahra Rajaei ¹, Shekoufeh Kolahdouz-Rahimi ^{1,2,*} and Massimo Tisi ³

¹ Department of Software Engineering, University of Isfahan, Isfahan, Iran; z.rajaei@eng.ui.ac.ir

² School of Computing, Engineering and the Built Environment, University of Roehampton, London, UK; shekoufeh.rahimi@roehampton.ac.uk

³ IMT Atlantique, LS2N, Nantes, France; massimo.tisi@imt-atlantique.fr

* Correspondence: shekoufeh.rahimi@roehampton.ac.uk; sh.rahimi@eng.ui.ac.ir (S.K.-R.)

Abstract

Graph learning leverages deep neural networks to analyze and infer from graph-structured data, to extract meaningful patterns and insights from graph data for various classification and prediction problems. The community is exploring ways to exploit graph learning to address several tasks within the field of Model-Driven Engineering (MDE). However, effectively using graph learning requires MDE practitioners to have a strong grasp of the technical details behind the graph-learning process, for instance to properly encode models in an input format that best suits the given graph neural network. In this work, we propose MoEGL, a domain-specific language designed to specify and perform the encoding process. The user names, in the metamodel's own terms, which classes, references and attribute values enter the graph. The tool writes NetworkX graphs and PyTorch Geometric heterogeneous graphs, and it can keep the type information of the model. On the graphs published by López et al., one configuration reproduces every graph we checked: 500 Ecore, 500 RDS and 369 Yakindu. On the 500 Ecore models a first run is faster than their Java encoder, and each task is 14 to 29 lines of YAML.

Keywords: Model-Driven Engineering; model encoding; graph learning; machine learning; pre-processing

1. Introduction

Model-driven Engineering (MDE) is a software engineering paradigm that centers models as the core artifacts of the development process [1]. Designers leverage models to define the structure and behavior of software systems using notations such as UML. MDE emphasizes models and model transformations to raise the level of abstraction and automate development activities [2]. Studies have shown that MDE approaches enhance the effectiveness and efficiency of software engineering in numerous situations [3]. However, barriers such as technical challenges, domain complexities, tooling and implementation difficulties, as well as social and community challenges have limited broader adoption of MDE.

Machine learning (ML) is a subset of artificial intelligence (AI) where systems learn from data to identify patterns and make decisions with minimal human intervention [4]. Recently, employing ML approaches for some MDE tasks has gained researchers' interest. For instance, Nguyen et al. [5] propose using neural networks to classify the metamodels into domain categories. Osman et al. [6] propose classifying UML models by extracting

Received:

Revised:

Accepted:

Published:

Copyright: © 2026 by the authors.

Submitted to *Technologies* for possible open access publication under the

terms and conditions of the [Creative Commons Attribution \(CC BY\)](#) license.

important features. Barriga et al. [7] propose automatically repairing software models via reinforcement learning. While progress has been made, ML's full potential for MDE tasks and modeling remains untapped. Our work builds upon these foundational studies by introducing MoEGL, a domain-specific language specifically designed to bridge the gap between model-driven engineering and graph neural networks (GNNs). Unlike previous approaches that primarily focus on homogeneous graph structures or manual encoding methods, MoEGL automates the encoding process and supports heterogeneous graphs, preserving the rich type information inherent in model artifacts. *Whether that extra information improves a downstream learner is a measured question, answered in Section 6.6, not an assumption.*

In general, ML models do not accept raw modeling artifacts as input. The phase of preparing data to be given as inputs to a ML model is often referred to as *data pre-processing*. This phase involves transforming the data into a format suitable for input into the ML model. This typically includes steps such as parsing the files, extracting relevant information, representing the data as a suitable data structure (e.g., tree, graph), and encoding it into numerical or categorical features that can be fed into the neural network. In the paper, we use the term *Model Encoding* to refer to these steps when applied to modeling artifacts. Note that in addition to encoding, pre-processing may involve handling any necessary data cleaning, normalization, or augmentation to improve the quality and usability of the input data for the ML model.

Today, every work that applies graph learning to models writes its own encoder: a program, typically a few hundred lines of Java or Python, that loads the models with EMF or PyEcore, decides which objects become nodes and which references become edges, encodes attribute values and serialises the result for a graph-learning library. Such encoders are specific to one metamodel and one learning task, are rarely reused, and changing the encoding (e.g., adding a reference, dropping a class, or encoding names) means changing code.

Our research addresses this gap by introducing a Domain-Specific Language (DSL) for *model encoding*, i.e., for specifying declaratively how a set of models is turned into input graphs for a GNN. The user writes a short configuration in terms of the metamodel (which classes, references and attributes to keep, and how to encode each attribute value), and the tool performs the loading, filtering, graph construction and export. The intended benefit is to move the encoding effort from every project into one reusable tool, so that a new encoding is a change in a configuration file rather than a new program.

We evaluate this proposal with the three research questions stated in Section 2.1: whether MoEGL configurations reproduce exactly the graphs of existing hand-written encoders (fidelity), what they cost in specification size and running time compared with those encoders (cost), and whether the two features that go beyond prior encoders, heterogeneous graphs and attribute encoding, improve the results of a real learning task (effect of encoding choices).

Several representations for modeling artifacts are proposed in literature of ML for MDE, such as models as raw graphs [8], graph kernel [9], models as trees [10], and models as documents [5,11]. Transforming the model to a raw graph is a popular encoding solution as it can easily maintain structural information [12]. However, the encoding is time-consuming and error-prone and it often proceeds by trial-and-error.

The language is called MoEGL (Model Encoding for Graph Learning). A workshop paper proposed a DSL that feeds homogeneous graphs to a GNN [13]. This article adds heterogeneous graphs (Section 4.1) and attribute-value encodings (Section 5.2), and a running example based on state machines. The evaluation answers three questions: an edge-preserving match to every published graph we checked, a per-task configuration of

14 to 29 lines of YAML that is faster than the Java encoder on the 500 Ecore models, and a classification result in which relation tokens raise balanced accuracy from 0.808 to 0.824 (Sections 6, 6.5 and 6.6).

The paper is organized as follows. Section 3 briefly describes concepts and tools used in our encoder. Section 4 describes our encoding process. Section 5 describes the MoEGL language. Section 6 evaluates the application of MoEGL. Section 7 outlines related work, and Section 9 concludes the paper.

2. Research Objectives, Contributions, and Questions

The three questions are stated in this section. The introduction names them once and points here. Section 8 names the experiment that answers each question.

2.1. Research Questions

The introduction and this section use one set of questions. Each question names the experiment that answers it. We ask about model encoding, that is, how a model becomes a graph, and not about model transformation in the sense of producing another model.

- **RQ1 (Fidelity).** Does a MoEGL configuration reproduce the graphs of an existing hand-written encoder? Answered by Experiment 1 on the three Lopez et al. corpora (Section 6).
- **RQ2 (Cost).** What does that configuration cost in specification size and running time, compared with the hand-written encoder? Answered by the measurement protocol in Section 6.5.
- **RQ3 (Effect).** Do heterogeneous graphs and attribute encoding change the result of a real learning task? Answered by the model-classification experiment in Section 6.6.

3. Background

This section keeps only the notions the encoder uses.

3.1. Graph Neural Networks (GNNs)

A GNN computes a vector for each node by passing messages along edges [14,15]. The same network can be asked for a decision about a node, an edge, or the whole graph [16]. MoEGL writes the graph that such a network reads.

3.2. Models as Graphs

An EMF model is already a graph: each object is a node and each reference is a directed edge [1]. The node type is the object's class and the edge type is the reference name. Figures 1–4 are the running example: a finite-state-machine metamodel, the graph of that metamodel, one instance, and the graph of that instance.

3.3. Heterogeneous vs. Homogeneous Graphs

A homogeneous graph has one node type and one edge type. A heterogeneous graph has several of each [17]. A model with more than one class is heterogeneous in this sense. Section 6.6 compares the two on the same node features.

3.4. PyEcore

PyEcore is a Python implementation of EMF [18]. An earlier MoEGL loader used it. The loader timed in Section 6.5 reads XMI and Ecore with lxml and does not call PyEcore.

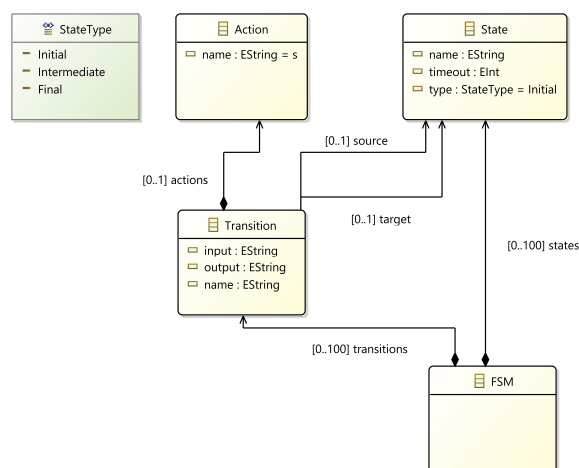


Figure 1. The ecore diagram of a FSM.

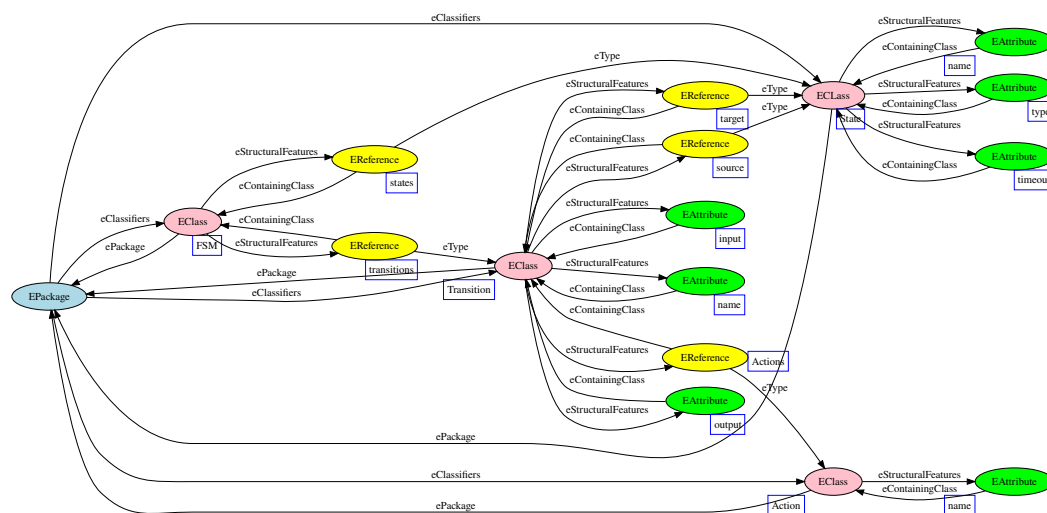


Figure 2. The corresponding graph of the metamodel in Figure 1.

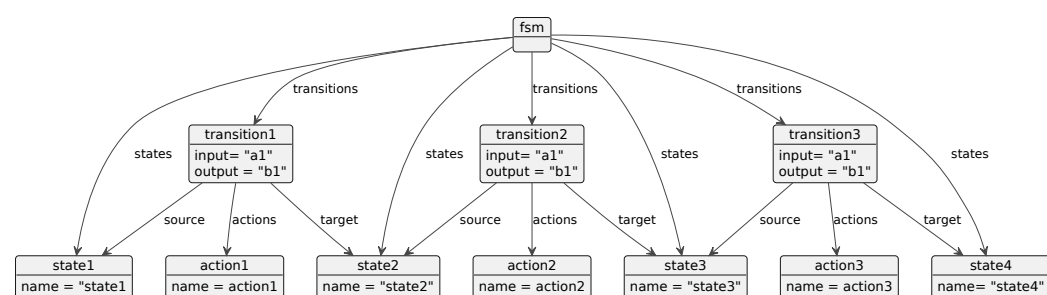


Figure 3. An instance model of the FSM in Figure 1.

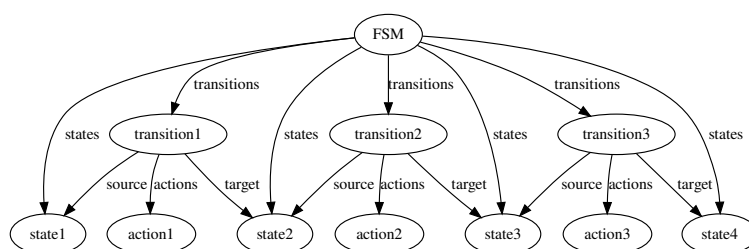


Figure 4. A graph corresponding to the instance model in Figure 3.

4. Encoding Process

The architecture of our proposed encoding of models is illustrated in Figure 5. It consists of three main technical spaces: model engineering, data gathering, and graph presentation technical spaces. In the first phase the current loader reads the models with lxml into an in-memory cache. An earlier loader used PyEcore; that is not the loader timed in Section 6.5.

In the next phase, the MoEGL configuration is loaded to check for necessary adaptations. The MoEGL configuration, written by the user, conforms to the class diagram in Figure 6. It is described in more detail in the next section. The configuration then selects classes and features from that cache. The data are organized into three levels including node data, edge data, and attribute type data. For example, the organization of the node and edge data for the model in Figure 1 would be as follows: The node data contains one EPackage file with the EPackage data, four EClass files with the EClass object data, five EReference files with the EReference object data, and seven EAttribute files with the EAttribute objects data. The edge data files contain EPackage–EClass edges, EClass–EPackage edges, EClass–EReference–EClass edges, EClass–EReference edges, and so on.

Finally, the data extracted from the organized files are transformed into an in-memory representation according to the user needs specified in the MoEGL configuration. We provide two output graph formats, including the NetworkX and Heterogeneous PyTorch Geometric (HPyG) graphs. MoEGL is described in more detail in the following section.

Existing works in the literature employ NetworkX for representing models as homogeneous graphs. We add support for mapping models to heterogeneous graphs. These graph types are more precisely aligned with the model structures, by supporting node types associated with sets of attributes, thus allowing object attributes to be directly represented.

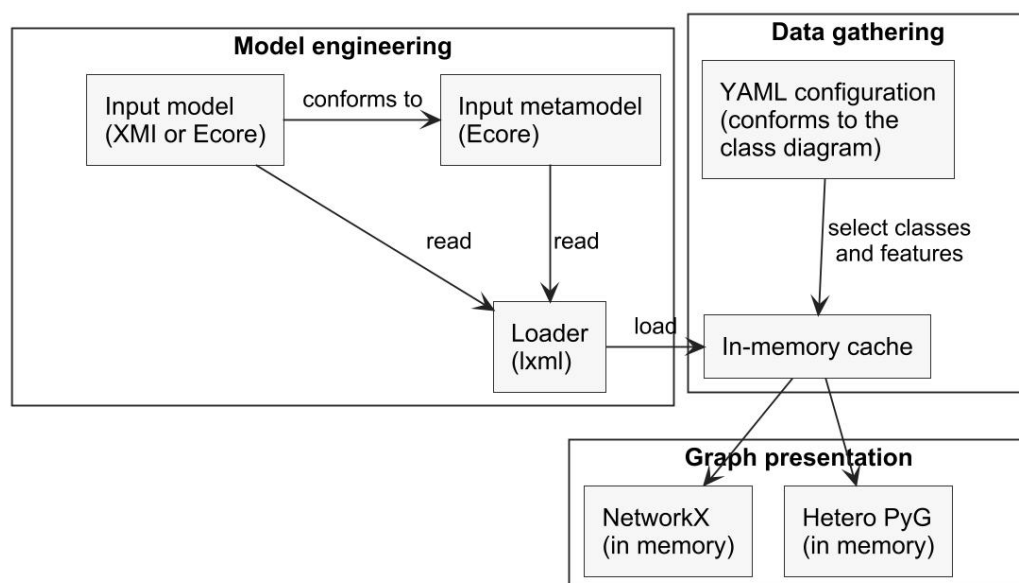


Figure 5. The architecture of the proposed process.

4.1. Heterogeneous Graph Construction

A heterogeneous graph is a general graph model that can represent multi-typed entities and relations. As defined by Sun et al. [17], a heterogeneous graph G is represented as $G = \{V, E, \phi, \psi\}$ where V is the set of nodes, E is the set of edges, $\phi(v)$ assigns a type to each node v , and $\psi(e)$ assigns a type to each edge e . More specifically, $T_v = \{\phi(v) : \forall v \in V\}$ contains the distinct node types defined in the graph schema, and $T_e = \{\psi(e) : \forall e \in E\}$ contains the distinct edge types. This typing of both nodes and edges allows a heterogeneous graph to represent diverse semantic entities and their interconnectivity in a structured manner.

The graph degenerates into a regular homogeneous graph when $|T_v| = |T_e| = 1$, meaning there is only one node and one edge type.

Here, we undertake an in-depth examination of the encoding technique applied to various input models, such as metamodels and instance models, comparing this encoding approach to representations utilized in prior work. Considering the metamodel of the FSM in Figure 1, it contains 1 node of type “EPackage”, 4 nodes of type “EClass”, 5 nodes of type “EReference”, and 7 nodes of type “EAttribute”. As illustrated in Figure 2, nodes sharing an equivalent color within the graph possess an identical type, attribute set, and relationship configuration. When metamodels are encoded as graphs, node attributes serve to represent the properties belonging to each object (referred to as metadata). For instance, an “EClass” node might contain an “abstract” attribute, whereas an “EReference” node could outline attributes such as “lowerBound”, “upperBound”, “ordered”, “unique”, “required”, “changeable”, and “volatile”. These properties aid tasks involving the architectural structure of metamodels, including but not limited to model generation, completion, and ML applications.

Considering the instance model depicted in Figure 3, it contains four types of nodes (FSM, Transition, State, and Action) and five types of edges (transitions, states, source, target, and actions). Additionally, the individual nodes themselves carry substantial attribute information (e.g. name, input, output, type, and timeout). Furthermore, the various relationships between the nodes convey richer semantic information. Due to the diversity of node and edge types present, a homogeneous graph alone is unable to represent all facets of the information within the model. When handling nodes and relations with different semantics in such models, graphs originally proposed using homogeneous structures may encounter notable obstacles. In previous work, node types in homogeneous graphs have been managed by defining a “type” attribute for each node. HGNNs are able to learn on attributes with different types stored for each node type, effectively handling the inherent complexity of heterogeneous graphs. This complexity arises because, unlike homogeneous graphs where all nodes share the same set of attributes, heterogeneous graphs contain multiple types of nodes and edges, each with unique attributes and relationships. HGNNs address this challenge by utilizing specialized mechanisms to process and integrate these diverse attributes and structural information. They employ tailored aggregation functions to capture the semantic and structural information unique to each node type, ensuring that the rich semantic and structural diversity of heterogeneous graphs is preserved and leveraged effectively for various applications.

5. MoEGL: Encoding Configuration DSL

We propose MoEGL, a DSL to flexibly encode domain models as graphs tailored to users’ needs. We defined a list of requirements for MoEGL, that we used to drive its design. We list them here. [Section 6 answers the three research questions of Section 2.1: fidelity, cost, and the effect on a learner.](#)

- MoEGL syntax must be simple, clear, and unambiguous to avoid any misinterpretations. Static checks on model elements are also important. This includes ensuring the uniqueness of class and feature names within the same package.
- The language must be familiar and intuitive for users. Using YAML, a widely known data serialization format, can help with this.
- There also needs to be a strategy to prevent naming collisions between generic encoding properties and meta-model elements
- All EMF metamodels (including cyclic ones) have a hierarchical containment structure. We want to organize MoEGL based on this structure, which is already familiar to EMF practitioners. This way we can also statically assess consistency with the EMF

metamodel, and its coverage. As a consequence of this choice our handling of cycles is limited, as we can copy them or remove them, but we can not create new cycles. This is however sufficient for our needs, since the encoding process does not generally modify the structural properties of the graph.

- It is crucial to support different output formats suitable for diverse user needs.
- The language must allow access to all meta-model elements and features via YAML sections.
- Crucially, MoEGL needs options to flexibly adapt the meta-model structure as required. For example, renaming model elements or encoding attribute values. This level of customization is key to meeting varied user requirements.

These items are the design aims. The evaluation measures how far the language meets them: the reproduced graphs, a configuration of 14 to 29 lines, the running time against the Java encoder, and the classification result. A study with practitioners is future work.

We implement our language as a schema defined within the YAML format. YAML [19] serves as a data serialization framework. It incorporates various segments such as structural details and raw informational payloads. YAML presents a relatively straightforward usage while maintaining compatibility with many common programming languages like Python. The relationships, characteristics, and organization of MoEGL are visualized conceptually through the class diagram presented within Figure 6. This diagram outlines the primary abstractions, their attributes, and the connections between these abstractions that underlie the structure and functionality of the language that has been constructed to serve the needs of the targeted domain. The structure of possible sections and values in YAML is shown in Listing 1. All the code implementations of the proposed DSL are available on GitHub at <https://github.com/Z-Rajaei/MoEGL>.

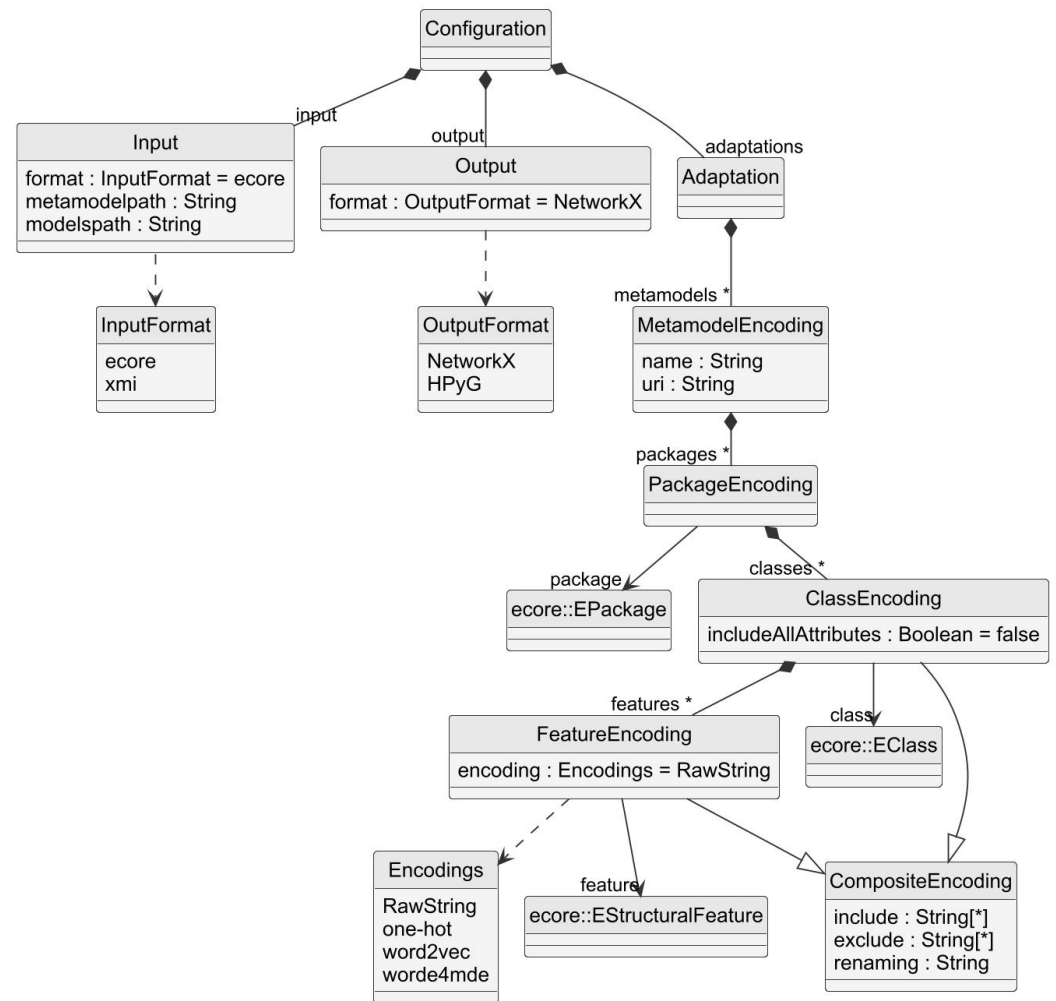


Figure 6. The class diagram of MoEGL.

```

input:
  format: "xmi"
  metamodelpath: "examples/running_fsm/fsm.ecore"
  modelspace: "examples/running_fsm/models"
output:
  format: "NetworkX"
adaptations:
  metamodels:
    packages:
      FiniteStateMachine:
        uri:
          www.fsmmetamodel.com:
            classes:
              include: ["FSM", "Transition", "State"]
              exclude: ["Action"]
              includeAllAttributes: false
            FSM:
              include: ["transitions", "states"]
              features:
                transitions:
                  renaming: "trans"
            Transition:
              include: ["source", "target", "name"]
              features:
                name:
                  encoding: "RawString"
  
```

LISTING 1: The MoEGL syntax, as a configuration that the tool loads and runs on the running example.

Listing 1 is the syntax of every configuration in this paper. The three top-level keys are `input`, `output` and `adaptations`. `input.format` is `ecore` or `xmi`. `input.metamodelpath` is the metamodel file and `input.modelspath` is the directory of models. `output.format` is `NetworkX` or `HPyG`. Each entry of `adaptations.metamodels.packages` is a package name whose `uri` selects the package, and the `classes` object under that URI holds the filters. `include` and `exclude` name classes or, inside one class, features. `includeAllAttributes` is a Boolean. A class is configured under its EClass name: `renaming` changes that name in the graph, and `features` names a feature, which may be an attribute or a reference: `renaming` applies to either, and `encoding` applies to an attribute value. The encoding is one of `RawString`, `word2vec`, `wordEmbed` and `one-hot`. Paths are relative to the MoEGL package root. Listing 1 and the three YAML listings that follow it were loaded and executed on the files in `examples/running_fsm`. The experiment listings are the configurations already used for Tables 1 and 2.

The three sections of a configuration are `input`, `adaptations` and `output`, as written in Listing 1. The “adaptations” section allows for customizing how the elements from the metamodel are encoded as nodes and edges in the graph. This preprocessing is necessary to transform the models into a format that is suitable for various ML tasks. Each adaptation incorporates one or more “MetamodelEncoding” components empowering the user to get to the intended metamodel through the “name” and “URI” addresses of its package. Under each “PackageEncoding” are “Class Encoding” components that allow the user to selectively include or exclude which model element classes will be represented as nodes in the final encoded graph. Individual classes can then be further configured via their name, where the features of each class can be selectively included or excluded at the attribute and reference level. The user can also specify whether all class attributes participate in encoding using “includeAllAttributes”. We do not need to provide OCL constraints, since we define a policy to unambiguously handle every combination of these operators. In our policy, attribute exclusions have the highest priority, followed by attribute inclusions. For instance, if “includeAllAttributes” is followed by an “exclude”, the DSL will include all the attributes that are not explicitly excluded.

The capabilities of the MoEGL configuration allow users to selectively determine which model aspects are included or excluded in/from the final representation. Individual characteristics can be included or excluded from the final encoded graph structure by utilizing their given names or labels in conjunction with the “include” and “exclude” directives. Features can be included or excluded by name using “include” and “exclude”. A feature can be an attribute or a reference. If the excluded feature is a containment reference, the program simply removes the reference and not the target class. Therefore, if we also need to remove the target, it should be excluded explicitly by the “exclude” option in the class section. When a containment reference is excluded, the program removes the reference because it represents the relationship or linkage between elements rather than the elements themselves. The target class, being a separate entity, is not automatically removed to preserve the integrity of other potential references or usages within the model. If the target class were to be removed automatically, it could inadvertently affect other parts of the model that rely on it. Therefore, to ensure precise and controlled modifications, the user must explicitly exclude the target class in the class section. This approach maintains the model’s consistency and prevents unintentional deletions that could disrupt the overall structure and functionality of the model.

Each class offers attribute configuration. Similarly, the user can access classes as shown in the diagram. Renaming modifies element names for tasks involving the comparison of metamodels with identically named but distinct attributes. For instance, the user can specify in the DSL that a “surname” attribute should be renamed to “lastname”, so that

it will share the same encoding of another “lastname” attribute in the metamodel. The feature encoding section specifies how the values of attributes and references should be represented, such as raw strings or using vector representations obtained via techniques like word2vec. Further explanation is provided regarding these encoding representations and techniques later on. The “input” meta-schema specifies the data format of source models, as the declarative language is designed to encode models defined in both Ecore and XMI formats. Relevant file paths must also be set for both the metamodel and models. The desired output graph format such as NetworkX or HPyG is then configured within the output section settings. Proper configuration of input data retrieval and desired output formatting is crucial for effectively transforming models into machine-readable graph structures while preserving original semantic meaning.

The declarative schema language therefore provides access and manipulation capabilities applicable to full metamodels, individual model classes, and even single attributes or references. The second major processing stage depicted in Figure 5 relies on these pre-encoding adaptations and configurations by way of a specified Python method to selectively check and include only relevant model aspects.

5.1. *Semantics of a Configuration*

The list at the start of this section states the design aims. The rules below are the behavior of the loader and of the adaptation step. A configuration does not receive a coverage score.

- **Coverage.** With no class include list, every class is kept except those named by exclude. With an include list, a class is kept when its name, or the name of one of its supertypes, appears in that list. exclude is applied first, so a class named in both lists is dropped. A dropped class removes its objects and every edge that touches them. Inside a kept class, exclude is again applied first. If any applicable class section gives an include list, a feature is kept only when its name is in the union of those lists. If no such list is given, every reference is kept, and an attribute is kept only when includeAllAttributes is true. Dropping a feature removes that attribute or that edge. The target object stays until its own class is excluded. A rule written on a supertype applies to its subtypes.
- **Conflict resolution.** The two conflicts are a name present in both include and exclude, and two sections that rename the same element. Exclusion wins in the first case, for classes and for features. In the second case the renaming is taken from the most specific class section that defines it: the object’s own class before a supertype. Feature renaming uses the same order.
- **Identity and cross-resource references.** Each object receives an integer identity in the order the loader creates it. An edge stores those integers. A reference token is resolved inside the same file: a fragment that starts with / walks the containment tree, and any other token is an xmi : id or an attribute marked id. A token of the form uri #fragment is kept as an edge inside this file in two cases: the URI names this file, or the fragment still resolves here after the resource was renamed, in which case the URI is recorded in the load cache as repaired. An absolute URI, and a URI that starts with platform: or pathmap:, produces no object and no edge. A relative URI whose file exists beside the model is treated the same way: the external resource is out of scope. A token that contains # and does not resolve is stored as an attribute whose name is the feature name plus Name (for example eTypeName), holding the name after #. A root or a child from a foreign namespace is skipped.
- **Cycles.** Containment is walked as a tree, in document order, and each containment child receives an edge. A non-containment reference becomes an edge to an object that

already exists, so a cycle that the file already contains is copied. Filtering drops an edge by dropping one of its ends or by excluding its feature. The one edge added beyond the serialized references is the opposite named by the metamodel: when a feature records an opposite, and that opposite feature exists on the target class and is not derived, the loader writes the reverse edge under the opposite's name. On the Ecore metamodel used for these experiments, `EReference.eOpposite` records no opposite, so an `eOpposite` edge is written only where the file serializes it. No configuration key inserts any further edge.

- **Multi-edges.** The `NetworkX` graph is a `MultiDiGraph`. The loader stores at most one edge for each triple of source identity, target identity and feature name, so a second value of the same feature toward the same object does not create a second edge. Two features between the same pair remain two edges, distinguished by the edge key type. A multi-valued feature toward distinct objects is one edge per object.
- **Edge attributes.** The only key stored on an edge is `type`, and its value is the reference name, after renaming. The encodings `RawString`, `one-hot`, `word2vec` and `wordE4mde` apply to attribute values on nodes. A feature that is itself named `type` collides with the `NetworkX` node key of that name, and that writer raises an error. The heterogeneous writer keeps the class as the node type and stores the value in the feature tensor. Section 6.7 repeats this boundary.

5.2. Encoding Attribute Values

Numerical attributes are stored as their raw values unless the configuration says otherwise. For a string, the configuration names one of three encoders: a `word2vec` model fitted on the dataset being encoded, the pretrained `WordE4MDE` vectors [20], or a one-hot vector.

Names are split before either embedding is applied. Non-alphanumeric characters are separators, and a camelCase or PascalCase boundary is a separator, so `hasCustomer` becomes the tokens `has` and `customer`. Digits stay with the token they are attached to until a separator. The vector of a value is the mean of the vectors of its known tokens.

The local `word2vec` model is a skip-gram `Word2Vec` fitted once on every token of the dataset (dimension 64, window 3, minimum count 1, 20 epochs, one worker, seed 42). A value with no token, or whose tokens are all absent from that vocabulary, becomes the zero vector. On one small instance the training text is a handful of names, and the coordinates stay near the random initialisation, whose scale is $0.5/d$. The previous version of this article printed such a vector, with entries around ± 0.01 , from the running example. That vector was not a trained embedding, and we have removed it. A locally fitted model is useful when the dataset is large enough to estimate co-occurrence; the running example is not such a dataset.

That fit, and the vocabulary of a one-hot encoding, is built on every model passed to it. When the encoded values are later used as features of a supervised split, the fit has to be called on the training fold alone, and the fitted model is then applied to the validation slice and to the test fold. A token absent from the training vocabulary stays the zero vector. Fitting on the whole corpus before the split would place test names inside the vectors. Encoding a dataset that has no held-out split, such as the timed re-encoding in Section 6.5, uses the same fit on the whole set.

`WordE4MDE` is a skip-gram model trained by its authors on a large corpus of element names from real models (300 dimensions) [20]. `MoEGL` loads those vectors and applies the same tokeniser. A name whose tokens are all unknown becomes the zero vector. In the classification experiment of Section 6.6, a linear SVM on the mean `WordE4MDE` vector of each model scores 0.808 ± 0.024 balanced accuracy, level with the TF-IDF baseline on

element names, and a GIN that uses the same vectors as node features scores 0.770 ± 0.029 . We therefore provide WordE4MDE for datasets too small to fit word2vec. On this classification task it matches the bag-of-names baseline.

Categorical attributes that describe element or relationship types can be encoded using one-hot encoding. This involves mapping each unique category to a binary vector, with a 1 in the index of the present category and 0s elsewhere. One-hot encoding transforms categorical data into a numerical format while retaining the distinction between different categories. This encoding supports various predictive tasks involving element types.

We were also able to implement one-hot encoding for the same reason, as we have the list of possible values for each attribute. This method also provides an additional benefit, allowing the user to parse and run the models once, performing any desired type of encoding or customization on the models multiple times. Figure 7 illustrates the described process. The structure and attributes of each model are derived and extracted from the models. The final graph contains both the relational framework of the associations between models as well as the numerically encoded attribute values appointed to each individual node. Once completed, the graphs are then input as data into a GNN system. The neural network will utilize the combined structural and characteristic information from the graph to discern relationships and perform some analytical task, such as classification or prediction, based on the features and connections within the graphical portrayal of the original collection of models.

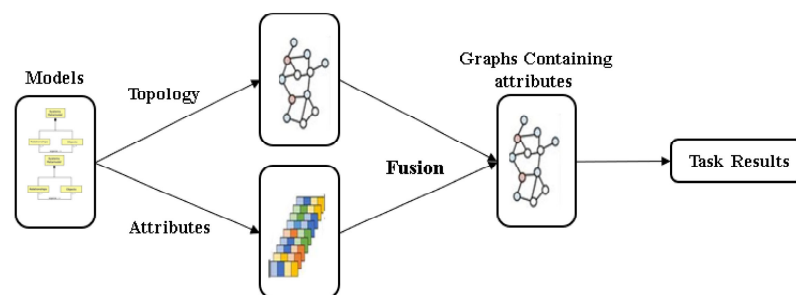


Figure 7. Diagram of extracting model structure and attributes for final graph.

5.3. Technical Implementation of MoEGL

In this section, we describe the detailed implementation process of MoEGL, highlighting the key steps involved in transforming models into graph structures suitable for Graph Neural Networks (GNNs). Our approach follows a structured process that includes model traversal, application of filtering rules, **encoding of attribute values on the nodes**, and the construction of both homogeneous and heterogeneous graphs.

1. Model Traversal and Element Extraction. The first step in our pipeline is the traversal of the input models, which can be provided in either ecore or xmi formats. Depending on the format, we load and traverse the models to extract key elements, such as nodes and edges. During this traversal, the model elements are systematically processed, and for each element, we capture its attributes and references. This hierarchical traversal ensures that all relevant model elements are visited and prepared for inclusion in the resulting graph structure.

2. Node and Edge Creation. As the model elements are traversed, we generate nodes and edges corresponding to each element and its relationships. Nodes are created for each class instance in the model, with their attributes stored for later use in encoding. Similarly, edges are generated to represent the relationships between different nodes, reflecting the references in the original model. This process ensures that the structure of the input model

is accurately preserved in the graph, maintaining the hierarchical and referential integrity of the data.

3. Application of Filtering Rules. To provide flexibility in model-to-graph transformation, we apply user-defined filters from the configuration file during the graph construction process. These filters allow users to specify which classes and attributes should be included or excluded from the final graph. The filtering is performed in two stages:

Node Filtering: Nodes are included or excluded based on the classes they represent, as defined in the filters. Users can also specify whether all or only certain attributes of a class should be included. Edge Filtering: Edges connecting nodes are filtered similarly, ensuring that only the relationships of interest are represented in the final graph. This customizable filtering mechanism allows users to tailor the graph representation to their specific needs, reducing complexity and focusing on the most relevant parts of the model.

4. Feature Encoding. Once the nodes and edges are filtered, we proceed with encoding their attributes into formats suitable for machine learning. Depending on the user's configuration, various encoding techniques are applied, including:

One-Hot Encoding: Categorical attributes are encoded using one-hot vectors, which is useful for representing discrete variables. Word2Vec Encoding: For textual attributes, we employ Word2Vec embeddings to capture semantic relationships between words in the model. This provides a dense, continuous representation of text data. Custom Embeddings (word4mde): For domain-specific terms in the model, we use a custom embedding technique that maps specific terms to pre-trained embeddings, ensuring that the domain knowledge is accurately reflected in the graph's features. [These encoding techniques transform attribute values stored on nodes into feature vectors that a GNN can read. An edge keeps the reference name as its type and does not receive one of these vectors. Section 6.7 states this limit together with the others.](#)

5. Graph Construction. After encoding the node and edge features, the graph is constructed. We support two types of graph structures:

Homogeneous Graphs: In this configuration, all nodes and edges are of the same type, which is useful for simpler models where there is no need to distinguish between different types of entities. Heterogeneous Graphs: For more complex models, we construct heterogeneous graphs, where nodes and edges can have multiple types. This allows us to preserve the distinct types of entities and relationships present in the model. The heterogeneous graphs are constructed using libraries such as PyTorch Geometric, which provides efficient handling of different node and edge types for machine learning tasks.

6. Output Formats. Finally, the constructed graphs can be output in various formats, depending on the user's requirements. We support both NetworkX (for homogeneous graphs) and PyTorch Geometric (for heterogeneous graphs). This flexibility allows the encoded graphs to be used directly in different GNN frameworks, enabling seamless integration with downstream machine learning pipelines. [The two writers emit an in-memory NetworkX graph and a PyTorch Geometric heterogeneous graph. CSV export, and reconstruction of a graph from CSV, are outside the tool. The tool also does not write the Deep Graph Library \[21\], a PyTorch Geometric InMemoryDataset, or Parquet. The time and peak memory in Section 6.5 are for the NetworkX encoding of the 500 Ecore models. Scalability past that set is stated there as unmeasured.](#)

5.4. Applying MoEGL to the Running Case

Assume that a user is working with a dataset containing thousands of FSM metamodel instances like the one shown in Figure 3. He wants to map all these models to NetworkX graphs for use in a ML task. However, there are a couple of constraints he wants to apply: 1) Exclude the "action" objects from the models, as they are not relevant to the

particular learning task he intends. 2) Rename the “transitions” edges to simply “trans” for consistency. The user defines the desired constraints and transformations in a YAML configuration file. As shown in Listing 2, only a few lines of code are needed to exclude the action objects and rename the transition edges. The adjacency matrix of the instance model in Figure 4 generated by applying the encoding of Listing 2 is shown in Matrix 1. The eight rows are the objects that remain after the listing drops every Action, in document order: the FSM, transition1, transition2, transition3, state1, state2, state3 and state4. A 1 means a directed reference from the row object to the column object; the matrix does not record the reference name. The first row is the FSM’s three transition edges and four state edges. The diagonal entry of that row is 0. The FSM class in Figure 1 has no reference to itself, and the object diagram has no such link. The previous version of this matrix had a 1 in that cell. Running Listing 2 on the instance file produces the matrix printed here, with 8 nodes, 13 edges and no self-loop. Now the user can run the learning tasks, and modify the YAML file to easily experiment with different preprocessing configurations, applying multiple “passes” over the dataset to determine the optimal representation for their ML task.

$$\begin{bmatrix} 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \quad (1)$$

Matrix 1: Adjacency matrix produced by running Listing 2. The diagonal is 0: the FSM object has no edge to itself.

```
input:
  format: "xmi"
  metamodelpath: "examples/running_fsm/fsm.ecore"
  modelspath: "examples/running_fsm/models"
output:
  format: "NetworkX"
adaptations:
  metamodels:
    packages:
      FiniteStateMachine:
        uri:
          www.fsmmetamodel.com:
            classes:
              exclude: ["Action"]
              includeAllAttributes: false
            FSM:
              features:
                transitions:
                  renaming: "trans"
```

LISTING 2: The configuration that produces Matrix 1. The class key is the EClass name FSM.

Let us examine in greater detail the process of configuring a more complex MoEGL program to encode the instance model. The user intends to encode the instance model in Figure 3, which includes various attributes of different types that must be appropriately encoded. The final goal is to train a neural network to automatically generate similar FSM models. Therefore, representing the attribute values accurately will be crucial to the network’s ability to learn.

The user has defined a MoEGL configuration to specify how each attribute should be encoded based on its data type. As shown in Listing 3, the MoEGL configuration outlines that string attributes will use word2vec embeddings to vectorize their values. On this running example the local word2vec fit has too little text to leave its initialisation, as explained in Section 5.2; the printed graph below therefore omits the coordinate dump. Meanwhile, the type attribute that can take on one of just three class labels will employ a one-hot encoding scheme. This involves transforming each class into a binary vector, with a 1 at the index of the class and 0s elsewhere.

Since the nodes of different types have different attributes and references, the graph representing the instance model must support heterogeneous node and edge types. For this purpose, the user has selected the HPyG library, which provides the necessary functionality. With attributes encoded via the prescribed methods, the graph will contain the rich information needed for the neural network to learn patterns. While NetworkX allows nodes and edges to have different attributes, it does not natively support multiple node and edge types as distinct entities. For this purpose, the user has selected the HPyG library, which provides the necessary functionality to handle heterogeneous node and edge types more effectively.

As a demonstration, Listing 4 is the heterogeneous graph produced by that configuration, not a separate figure and not a matrix. By examining this data, we can observe details like there being one FSM node, four State nodes, three Transition nodes, and three Action nodes in the graph. The various edge types connecting these nodes are also enumerated. Together, the encoded attributes and graph structure capture the key structural and semantic elements of the instance model for the neural network to analyze and potentially generate new similar FSM models.

```
input:
  format: "xmi"
  metamodelpath: "examples/running_fsm/fsm.ecore"
  modelspath: "examples/running_fsm/models"
output:
  format: "HPyG"
adaptations:
  metamodels:
    packages:
      FiniteStateMachine:
        uri:
          www.fsmmetamodel.com:
            classes:
              includeAllAttributes: true
              Transition:
                features:
                  name:
                    encoding: "word2vec"
                  input:
                    encoding: "word2vec"
                  output:
                    encoding: "word2vec"
              State:
                features:
                  name:
                    encoding: "word2vec"
                  type:
                    encoding: "one-hot"
              Action:
                features:
                  name:
                    encoding: "word2vec"
```

LISTING 3: Attribute encoding for the running example. The class is *Transition*, and every encoding key has its colon.


```

HeteroData(
    FSM={ x=[1, 1] },
    Transition={ x=[3, 192] },
    Action={ x=[3, 64] },
    State={ x=[4, 69] },
    (FSM, transitions, Transition)={ edge_index=[2, 3] },
    (Transition, actions, Action)={ edge_index=[2, 3] },
    (FSM, states, State)={ edge_index=[2, 4] },
    (Transition, source, State)={ edge_index=[2, 3] },
    (Transition, target, State)={ edge_index=[2, 3] }
)

```

LISTING 4: The heterogeneous graph produced by running Listing 3. Each x shape is (number of nodes, feature width). The State width 69 is a 64-dimensional name vector, the raw timeout and a 4-dimensional one-hot. For state1, type=Initial encodes as [0, 1, 0, 0] (vocabulary Final, Initial, Intermediate, plus an unknown bin) and timeout is the raw value 1. The name coordinates stay at the untrained initialisation of Section 5.2 and are not printed.

Listing 4 stores each relation as `edge_index` only, under the triple of source type, reference name and target type. That triple is the schema the writer emits. A meta-path is not constructed, and neither is a model that consumes one, such as HAN or MAGNN. Multiplicity and the containment flag stay on the node that declares the reference, when that object is kept in the graph. The HeteroData writer does not emit an edge-feature tensor. Section 6.7 states the same boundary.

```

input:
  format: "ecore"
  metamodelpath: ""
  modelspath: "examples/running_fsm"
output:
  format: "HPyG"
adaptations:
  metamodels:
    packages:
      ecore:
        uri:
          http://www.eclipse.org/emf/2002/Ecore:
            classes:
              includeAllAttributes: false
              EClass:
                include: ["name", "abstract", "eStructuralFeatures"]
                features:
                  name:
                    encoding: "word2vec"
                  abstract:
                    encoding: "one-hot"
              EAttribute:
                include: ["name", "eContainingClass"]
                features:
                  name:
                    encoding: "word2vec"

```

LISTING 5: A configuration that encodes the running-example metamodel. `eType` is a reference, so it stays an edge; the encoding keys apply to the attributes `name` and `abstract`.

We chose to represent our domain models using objects rather than arrays for improved conciseness and efficiency. For example, when an array stores objects, each element requires the same name property (e.g. “FiniteStateMachine”) to be repeated. During deserialization, this would necessitate mapping elements to infer relationships. In contrast, using an object allows each element to be uniquely referenced by name. This is particularly useful for our case where each model element encountered must be matched to determine what it corresponds to in the package and look up details. Listing 2 demonstrates this approach,

with the “packages” property containing an object with a “FiniteStateMachine” property. This in turn stores an object with “uri” and “classes” properties. Notably, “classes” is also an object rather than an array, with a property for each EClass name (FSM in Listing 2). Importantly, representing the model as an object graph eliminates potential naming collisions. Since the meta-model element names serve as the object keys, and these are guaranteed to be unique, there is no ambiguity in referencing elements during parsing or transformation. Overall, using objects allows our domain models to be encoded more concisely while enabling efficient lookup and disambiguation of elements during the serialization and deserialization process. The object structure directly reflects relationships in the domain model.

6. Experimentation

In this evaluation, we aim to demonstrate the expressiveness and usefulness of MoEGL for the model-encoding needs of the MDE community. To do so, we replicate several encoder implementations from the existing literature on applications of GNNs to MDE. By concisely specifying the mapping configurations in MoEGL, we can perform all the necessary model parsing, element matching, property extraction, and graph construction steps automatically. In this section, each test case will be inspected separately in a dedicated subsection, where the graphs for each one will be recreated using MoEGL.

6.1. Experiment 1

First, we validate MoEGL by replicating the experiments by Lopez et al. [8]. In [8], the authors train deep autoregressive models to generate realistic Ecore models. They use four datasets of real models: Ecore-GitHub, Yakindu-GitHub, Rds-GenMyModel, and Yakindu-Exercise. We use the Ecore-GitHub dataset, i.e., the 500 real Ecore meta-models released in the replication package of Lopez et al. (TCRMG-GNN, release 1.0.0), together with the 500 graphs that their Java encoder produced from these models and that are published in the same package. Their encoder keeps as nodes the objects of type EPackage, EClassifier (EClass, EDataType, EEnum), EStructuralFeature (EAttribute, EReference) and EEnumLiteral, drops annotations, operations, parameters and generic types, keeps nine references as typed edges (eClassifiers, ePackage, eStructuralFeatures, eContainingClass, eType, eOpposite, eSuperTypes, eLiterals, eEnum) and encodes no attribute values. Listing 6 is the complete MoEGL configuration that expresses this encoding. Class-level rules are subtype-aware, so listing EClassifier includes classes, data types and enumerations, and a rule written for EStructuralFeature applies to attributes and references.

MoEGL encodes the models into NetworkX multigraphs. Fidelity is certified by a node bijection that preserves every typed edge, obtained by colour refinement and checked edge by edge. This is not NetworkX’s default `is_isomorphic`, which compares only the shape of the graph and ignores node and edge attributes. Colour refinement starts from the node type, and the bijection is kept only when every typed edge is preserved. On the graphs published by Lopez et al. the certificate succeeds for all 500 Ecore graphs, all 500 RDS graphs and all 369 Yakindu graphs. The Yakindu directory contains 377 files; eight have no published graph, so they are not counted as failures. An earlier loader based on PyEcore matched 495 of the 500 Ecore graphs under NetworkX isomorphism. The running time in Section 6.5 is for the current loader. The five misses were loader bugs, not DSL bugs: EMF derives `eType` through the bounds of generic type parameters, which that loader did not implement (three models), and it set `eOpposite` symmetrically when a model declares it on one side only (two models). Re-running the authors’ own Java encoder on the released Ecore files reproduces 492 of 500, because eight released files reference themselves under

their original name. The current loader repairs those self-references. All comparison scripts are in the replication package.

```
input:
  format: "ecore"
  metamodelpath: ""
  modelspath: "realModels/Ecore"
output:
  format: "NetworkX"
adaptations:
  metamodels:
    packages:
      ecore:
        uri:
          http://www.eclipse.org/emf/2002/Ecore:
        classes:
          include: ["EClassifier", "EStructuralFeature",
                    "EPackage", "EEnumLiteral"]
          includeAllAttributes: false
          EPackage:
            include: ["eClassifiers"]
          EClassifier:
            include: ["ePackage"]
          EClass:
            include: ["ePackage", "eStructuralFeatures",
                      "eSuperTypes"]
          EEnum:
            include: ["ePackage", "eLiterals"]
          EEnumLiteral:
            include: ["eEnum"]
          EStructuralFeature:
            include: ["eContainingClass", "eType"]
          EReference:
            include: ["eContainingClass", "eType",
                      "eOpposite"]
```

LISTING 6: The complete MoEGL configuration that replicates the Ecore encoder of Lopez et al. [8] (29 lines of YAML).

6.2. Experiment 2

Rahimi et al. [22] presented an interesting approach for generating structurally realistic models using GANs in the context of MDE. The lack of freely available industrial models has motivated the development of model generation techniques. However, realism is often overlooked. Rahimi et al. applied GANs to create new models that closely resemble real ones. A tool is built based on the Eclipse Modeling Framework (EMF) and evaluates the generated models using graph-based metrics. Promising results showed the GAN could produce realistic instances. For validation, they used a dataset derived from a simplified Car Wash Management metamodel in Figure 8 containing five metaclasses (CarWash, Person, Car, Service, Bill). They created real instance models to form the dataset, which then trained the NetGAN to generate a new synthetic dataset for validation. NetworkX was used to encode the models as graphs. We obtained the eleven CarWash instance models (XMI) and the metamodel used by the authors and encoded them with a MoEGL configuration of 14 lines. Every class is a node type and every reference is an edge type; attribute values are omitted, as in the authors' encoder. The eleven graphs contain the five node types and the seven edge types of the metamodel (Table 1; 11 models, not the 10 and 20 that the previous text and table disagreed on). Listing 7 is MoEGL's encoding of one of those models. The authors did not publish a typed reference graph for these files. Their encoder builds an adjacency matrix and then discards direction and edge labels before NetGAN, so a typed-edge bijection against that matrix would compare two different graphs. We therefore report the encoding and the sizes, and we do not claim an isomorphism score for Experiment 2.

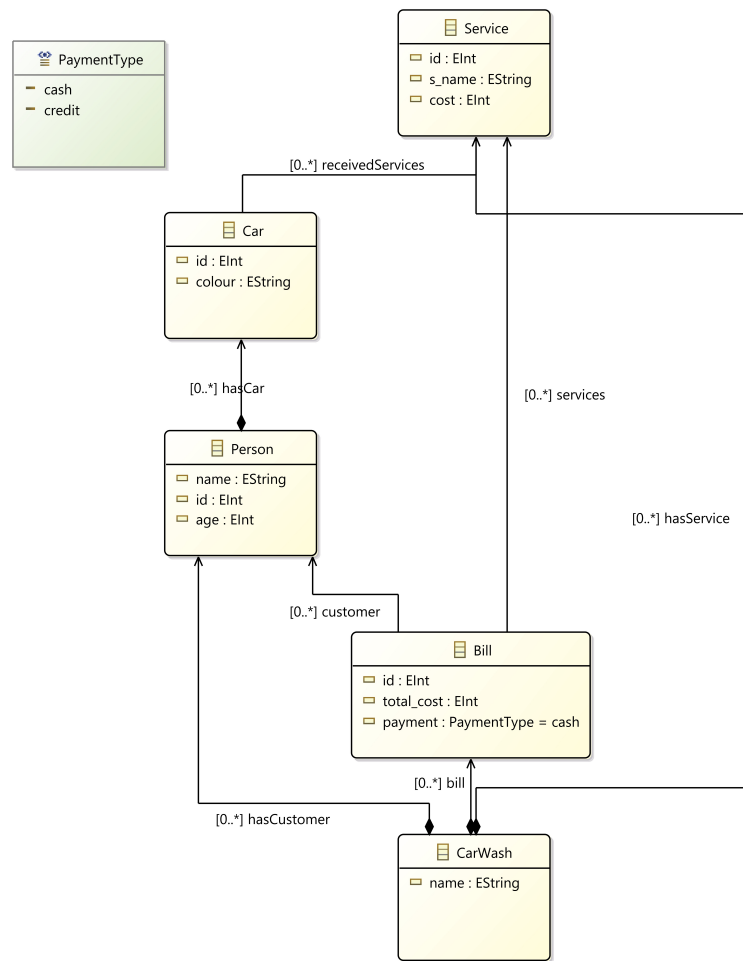


Figure 8. Carwash metamodel in [22].

```

{'directed': True, 'multigraph': True,
'graph': {}, 'nodes': [{ 'type': 'Bill', 'id': 0},
{'type': 'Bill', 'id': 1},
{'type': 'CarWash', 'id': 2},
{'type': 'Car', 'id': 3},
{'type': 'Car', 'id': 4},
{'type': 'Car', 'id': 5},
{'type': 'Person', 'id': 6},
{'type': 'Person', 'id': 7},
{'type': 'Service', 'id': 8},
{'type': 'Service', 'id': 9},
{'type': 'Service', 'id': 10}],
'links': [{ 'type': 'customer', 'source': 0,
'target': 6, 'key': 0},
{'type': 'services', 'source': 0,
'target': 8, 'key': 0},
{'type': 'services', 'source': 0,
'target': 10, 'key': 0},
{'type': 'customer', 'source': 1,
'target': 7, 'key': 0},
{'type': 'services', 'source': 1,
'target': 9, 'key': 0},
{'type': 'bill', 'source': 2,
'target': 0, 'key': 0},
{'type': 'bill', 'source': 2,
'target': 1, 'key': 0},
{'type': 'hasCustomer', 'source': 2,
'target': 6, 'key': 0},
{'type': 'hasCustomer', 'source': 2,
'target': 7, 'key': 0},
{'type': 'hasService', 'source': 2,
'target': 8, 'key': 0},
{'type': 'hasService', 'source': 2,
'target': 9, 'key': 0},
{'type': 'hasService', 'source': 2,
'target': 10, 'key': 0},
{'type': 'receivedServices', 'source': 3,
'target': 8, 'key': 0},
{'type': 'receivedServices', 'source': 4,
'target': 10, 'key': 0},
{'type': 'receivedServices', 'source': 5,
'target': 9, 'key': 0},
{'type': 'receivedServices', 'source': 5,
'target': 10, 'key': 0},
{'type': 'hasCar', 'source': 6,
'target': 3, 'key': 0},
{'type': 'hasCar', 'source': 7,
'target': 4, 'key': 0},
{'type': 'hasCar', 'source': 7,
'target': 5, 'key': 0}]]}

```

LISTING 7: The graph data of Carwash instance model in [22].

6.3. Experiment 3

In this experiment, we compared the graph encoding capabilities of MoEGL with the approach presented by Miranda [23], which used machine learning-backed methods to compute model views with Heterogeneous Graph Neural Networks (HGNNs). Miranda et al. aimed to simplify the integration of machine learning into Model-Driven Engineering (MDE) by encoding instance models conforming to an FSM metamodel into graphs using both NetworkX and Heterogeneous PyG. Their encoding approach was designed to automatically infer inter-model links while decoupling the contributions of view engineers and machine learning specialists.

Miranda et al. print one sample graph, reproduced as Listing 8: 3 nodes of type Transition, 4 of type State, and 6 edges whose types are the pairs (Transition, source, State) and (Transition, target, State). We compared that listing entry by entry. It is

one 7-node example, and it is the only graph from this paper for which a reference listing exists.

The MoEGL configuration for this encoding has 19 lines: it restricts the node classes to Transition and State, keeps only the source and target references of Transition, and encodes no attribute values. Table 1 is computed on a different file, the statecharts instance distributed with the authors' prototype. That file contains 101 objects, of which 100 are states or transitions, and 130 source/target links. It has no published reference graph, so the table reports its size and we do not call it a match to the 7-node listing. The configuration is in the replication package.

```
{
  "nodes": [
    {"type": "Transition", "id": 0},
    {"type": "Transition", "id": 1},
    {"type": "Transition", "id": 2},
    {"type": "State", "id": 3},
    {"type": "State", "id": 4},
    {"type": "State", "id": 5},
    {"type": "State", "id": 6}
  ],
  "links": [
    {"type": ("Transition", "source",
"State"), "source": 0, "target": 3},
    {"type": ("Transition", "target",
"State"), "source": 0, "target": 4},
    {"type": ("Transition", "source",
"State"), "source": 1, "target": 4},
    {"type": ("Transition", "target",
"State"), "source": 1, "target": 5},
    {"type": ("Transition", "source",
"State"), "source": 2, "target": 5},
    {"type": ("Transition", "target",
"State"), "source": 2, "target": 6}
  ]
}
```

L I S T I N G 8: The graph data of the FSM instance model in [23].

Table 1 summarises the model datasets of the three experiments and the graphs MoEGL produces from them. All numbers are computed by the replication scripts from the actual runs. “Objects” and “references” are counted on the loaded models before any adaptation (every object, every set non-derived reference link); “nodes” and “edges” are counted on the encoded graphs. The difference between the two pairs of columns is exactly what the configuration removes: in Experiment 1, annotations, operations, parameters and generic types (and their links) are dropped, so 61.4 objects become 43.9 nodes; in Experiment 2 nothing is removed, so the numbers coincide; in Experiment 3 only states, transitions and their source/target links are kept. Attribute values are not part of these graphs because none of the three original encoders uses them; the goal here is to evaluate whether MoEGL can replicate existing structural encoders.

The three encoders of prior work, written in Java and Python by three different groups, are each expressed by a MoEGL configuration of 14 to 29 lines. For Lopez et al., the only one of the three with a published reference corpus, the current loader reproduces every published graph (500 Ecore, 500 RDS, 369 Yakindu). This supports the claim that the encoding step can be delegated to a declarative configuration. The cost of doing so is quantified in Section 6.5, and the effect of the two features those encoders do not have, heterogeneous graphs and attribute encoding, is measured in Section 6.6.

Table 1. Datasets of the three experiments and the graphs produced by MoEGL (averages per model, computed from the actual runs; objects/references before adaptation, nodes/edges after encoding).

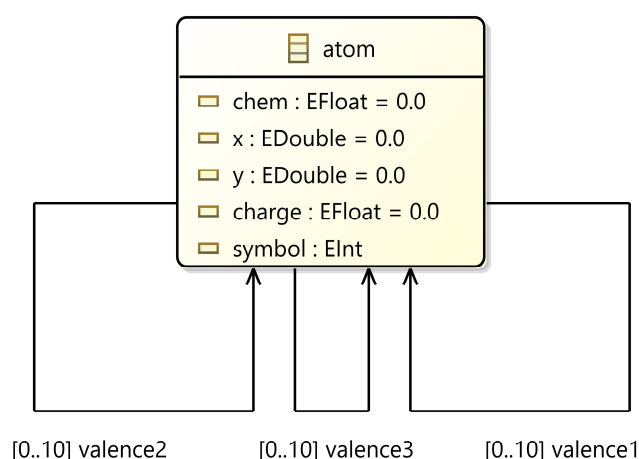
Experiment	Input	Models	Avg. objects	Avg. references	Avg. nodes	Avg. edges	Config. (LoC)
Exp. 1 [8]	ecore	500	61.4	159.6	43.9	120.5	29
Exp. 2 [22]	xmi	11	40.6	139.4	40.6	139.4	14
Exp. 3 [23]	xmi	1	101.0	266.0	100.0	130.0	19

6.4. Additional Experiment: Using Models in Graph-Learning Benchmarks

In the main experimentation, we replaced existing encoders from the MDE community with MoEGL encoders. Here we focus on a well-known graph-learning benchmark from the ML community, based on a graph dataset. We show that the same ML task can be performed using a dataset of modeling artifacts and a MoEGL encoder. This way, we want to show that MoEGL enables tackling common graph learning problems using models.

We choose an existing graph dataset and create a metamodel representing the graphs. We programmatically translate graphs of the dataset into instances of such metamodel. We write a MoEGL configuration that encodes these models. Finally, we compare the result of the encoding to the original graphs in the dataset, aiming at obtaining the same graphs. The AIDS dataset from TUDataset [24] is chosen as it contains node/edge labels and attributes. The dataset contains 2000 virus-screening compound graphs. Each node contains attributes like position, charge, and a label for its symbol. There are 38 symbol types and 3 edge types. The AIDS metamodel in Figure 9 depicts the structure of the AIDS graphs. The attribute values are stored in the node attributes vector. The model is converted to the NetworkX format using MoEGL. We compared the generated graph and the original graph and found that the encoded structure and attributes are the same.

We are aware that the experiment validity is hampered by a certain circularity (i.e. we create the metamodel and both the model-to-graph and graph-to-model transformation), but we believe it to be still a meaningful hint to the potential of broadening the applicability of MDE techniques to graph learning.

**Figure 9.** The metamodel representing the graphs in the AIDS dataset.

6.5. Evaluation of MoEGL against Bespoke Encoders

This section quantifies the cost of using MoEGL instead of a hand-written encoder along three measurable dimensions, encoding time, peak memory and specification effort (lines of code), and states explicitly which properties we do *not* measure. All numbers

are produced by the scripts of the replication package and can be regenerated with one command.

6.5.1. Measurement Protocol

Dataset. The Ecore-GitHub dataset of Experiment 1: 500 real Ecore metamodels (21,938 objects in total), the only one of the three datasets for which the original encoder and its published output are available, so that fidelity, time and code size can all be measured on the same input.

Compared encoders. (a) The bespoke encoder of Lopez et al. [8], i.e., the authors' own Java implementation (EMF 2.20, JGraphT, Gson) executed unchanged through a 40-line driver that performs exactly the steps of their `GenerateRealGraphsEcore` program on the 500 files (resource loading, graph construction, JSON serialisation) without the corpus filtering and random sampling that precede them, since the released files already are the sampled set. (b) MoEGL with the configuration of Listing 6, run as a Python process that performs Stage 1 (model loading with an `lxml` reader and extraction of objects and links into the cache) followed by Stage 2 (adaptation and `NetworkX` graph construction).

Timing. Each measurement is the wall-clock time of the encoding loop inside the process (JVM or Python interpreter start-up excluded), measured with `System.nanoTime` and `time.perf_counter`, respectively; we also report the total process time including start-up. Every configuration is executed in $N = 10$ independent, freshly started processes and we report the mean and the sample standard deviation over the 10 runs. In addition, we measure MoEGL's *re-encoding* time, i.e., Stage 2 alone starting from the Stage 1 cache with a second configuration (the same graph without data types and enumeration literals, and with class names encoded by `word2vec`), which is the situation of a user who iterates on the encoding of a fixed dataset. All runs were executed sequentially on an otherwise idle laptop: AMD Ryzen 9 4900H (8 cores/16 threads), 47 GB RAM, Windows 11 (build 26200), models on a local SSD, OpenJDK 21.0.3, Python 3.10.9, `lxml`, `NetworkX` 3.4.2, `gensim` 4.4.0.

Memory. Peak memory was measured in a separate session, with the same two commands, the same 500 models and the same machine: ten fresh Java processes, then ten fresh MoEGL processes. Every 20 ms the process tree is sampled and the operating-system peak counters are read. Those counters are cumulative, so the value is the high-water mark of the worker, not a single instant. The worker is `java.exe` for the bespoke encoder. For MoEGL it is the base `python.exe` interpreter; the virtual-environment launcher only starts that interpreter and is not the measured process. Peak working set is `PeakWorkingSetSize`, the most physical memory resident in the worker. Peak committed memory is `PeakPagefileUsage`, the most memory the worker committed, which for the JVM includes heap that was not all resident. Both are reported in MB, where $1 \text{ MB} = 2^{20}$ bytes, as the mean and the sample standard deviation over the ten runs. The 47 GB figure is the installed RAM of the laptop, not the memory used by either encoder.

Lines of code. Counted with `cloc` 2.06 (code lines only; blank and comment lines excluded). For the three bespoke encoders we count only the files that implement the model-to-graph encoding, taken from the authors' public repositories, and exclude training, generation and plotting code: for Lopez et al. the Java classes `Parser`, `GraphModel`, `MetaFilterLiterals`, `IMetaFilter` and the Python loader `json2graph.py` (409 LoC), plus the 190-LoC program that drives them; for Rahimi et al. [22] `encoder.py` (249 LoC); for Miranda et al. [23] `to_graph.py`, `encoders.py` and `metamodels.py` (193 LoC). For MoEGL we count the tool itself (655 LoC of Python, of which 68 are the command-line interface and 56 the optional `HeteroData` exporter) and, separately, the YAML configuration of each experiment.

6.5.2. Results

Table 2. Encoding the 500 Ecore models of Experiment 1: fidelity, time and peak memory (mean $\pm$ sample standard deviation over $N = 10$ fresh processes) and size of the specification. LoC counted with `cloc` (code lines only). Memory is in MB, with $1 \text{ MB} = 2^{20}$ bytes.

	Bespoke encoder [8] (Java)	MoEGL (Python)
Graphs identical to the published ones	492 / 500 ^a	500 / 500
Encoding time, in-process (s)	1.48 ± 0.10	1.11 ± 0.02
of which Stage 1: model loading and extraction	–	0.67 ± 0.01
of which Stage 2: adaptation and graph construction	–	0.45 ± 0.01
Total process time incl. start-up (s)	2.01 ± 0.12	1.81 ± 0.05
Peak working set (MB) ^c	236.6 ± 10.1	153.5 ± 0.2
Peak committed memory (MB) ^c	957.0 ± 10.8	624.6 ± 0.3
Re-encoding from cache with a second configuration (s)	n/a (full re-run)	1.16 ± 0.33 ^b
Code written once (tool), LoC	409 (+190 driver)	655
Code written per encoding task, LoC	409 (+190 driver)	29 (YAML)

^a The authors' code re-run on the released files; 8 released files reference themselves under their original name and load with unresolved links. ^b Includes training a word2vec model on all class names; with the original configuration Stage 2 alone takes 0.45 ± 0.01 s. ^c Separate session from the times, same commands. Peak working set is `PeakWorkingSetSize` of `java.exe` or of MoEGL's `python.exe`. Peak committed memory is `PeakPagefileUsage`. The process tree was sampled every 20 ms; the counters are cumulative.

Fidelity. The timed loader is the current one. An edge-preserving bijection matches 500 of 500 Ecore graphs, 500 of 500 RDS graphs and 369 of 369 Yakindu graphs (Section 6, Experiment 1). The graphs timed here are the same graphs as that certificate: after the loader changes in this section, the multiset of typed nodes and edges on the 500 Ecore models was unchanged. An earlier PyEcore loader matched 495 of 500.

Time. On a first complete run the current loader is slightly faster than the bespoke Java encoder: 1.11 ± 0.02 s against 1.48 ± 0.10 s for the same 500 models, about 2.2 ms per model against 3.0 ms. Stage 1, which reads the files with `lxml`, takes 0.67 ± 0.01 s and Stage 2 takes 0.45 ± 0.01 s. In this session every one of the ten MoEGL runs (1.09–1.14 s) was faster than every Java run (1.36–1.68 s). An earlier PyEcore loader took 7.98 ± 0.11 s on the same task, of which 7.58 s was Stage 1; that loader is not the one timed in Table 2. Re-encoding from the cache, including a word2vec fit, takes 1.16 ± 0.33 s, while the Java encoder re-parses every model on each change. Both times remain small next to training a GNN on the resulting graphs.

Memory. In the memory session the Java worker reached a peak working set of 236.6 ± 10.1 MB and a peak committed size of 957.0 ± 10.8 MB. The MoEGL interpreter reached 153.5 ± 0.2 MB resident and 624.6 ± 0.3 MB committed. Each figure is the high-water mark of one process that encodes all 500 models. It is not a per-model cost, and it is not the 47 GB of RAM installed on the machine. The committed size is larger than the working set because both runtimes commit memory that is not all resident at the peak; the gap is wider for the JVM.

Specification effort. A fair comparison must account for the fact that MoEGL itself is code: 655 lines of Python that are written once and shared by all encoding tasks, against 409 (or 599 with the driver) lines for the one bespoke encoder we could measure in full, 249 and 193 lines for the two others. The difference lies in the cost *per task*: each of the three encoders of prior work is expressed by 14 to 29 lines of YAML (Table 1), and a new encoding for the same models is a change of a few lines in that file. The claim we make is therefore not that MoEGL removes code, but that it moves the code from every project into one reusable tool, so that the per-task effort drops from a few hundred lines of Java

or Python, which require knowledge of EMF or PyEcore and of the graph library, to a few dozen declarative lines that only require knowledge of the metamodel.

6.5.3. Design Properties

Scalability, ease of use, maintainability and extensibility follow from the architecture, alongside the measurements above. *Scalability*: the two-stage design builds the cache of a repository once and re-uses it; on the 500-model set the loader reads the models in 0.67 s. *Ease of use*: a configuration names classes and features in the metamodel's own terms, in 14 to 29 lines of YAML. *Maintainability and extensibility*: a new encoding is an edit of that file, and a new value encoder is one function in the encoder module. A study with practitioners, and a timing series on corpora larger than these 500 models, are future work.

6.5.4. Threats to Validity

Construct validity. Lines of code is an imperfect proxy for effort; it does not capture the time needed to learn a tool or the difficulty of the code. We mitigate this by counting only code lines, by applying the same tool and rules to all artefacts, and by reporting the size of MoEGL itself rather than only the configurations. The isomorphism test compares graphs with node and edge types only, which is exactly the information the compared encoders produce. *Internal validity*. Timing compares a Java and a Python implementation and therefore also reflects language and library differences (EMF versus the lxml loader); we separate the loading stage from the DSL-driven stage to make this visible and we report both in-process and total process time over ten fresh runs. The memory figures have the same limit: they are the peak of the whole worker process, including the JVM or the Python interpreter, not the graph alone. The memory session is separate from the timing session. *External validity*. The timed comparison uses one dataset and one bespoke encoder. The edge-preserving certificate additionally covers the published RDS and Yakindu graphs. The two other encoders are compared on code size and on small datasets (11 and 1 models). The classification result is for one task and one near-duplicate split; it does not transfer automatically to other MDE tasks, and our split is not the unpublished duplicate list of Lopez et al. [25]. Behaviour on very large instance models has not been measured. We have not checked whether names from this ModelSet split occur in the corpus on which WordE4MDE was trained. The balanced accuracies 0.808 and 0.824 in Section 6.6 are the TF-IDF SVM and the relation-token SVM. *Reliability*. All scripts, configurations, the compiled driver for the bespoke encoder, the raw timing samples and the per-model comparison results are available in the replication package, together with the exact software versions used.

Robustness of a downstream learner. Tahmasivand et al. [26] detect and recover bit-flips inside a language model. Zahran et al. [27] jailbreak quantized language models by injecting faults into their weights. Omid et al. [28] measure the adversarial accuracy of convolutional networks trained on an undervolted GPU. These three studies concern hardware faults in language models and convolutional networks. None of them takes an EMF model or a MoEGL graph as input. This article does not measure bit-flip detection, jailbreaks under fault injection, or adversarial robustness under undervolting. The reliability it reports is the edge-preserving reproduction of the published graphs (Section 6) and the split protocol of Section 6.6.

6.6. Effect of the Encoding on Model Classification

Experiments 1–3 encode no attribute values, because the encoders they replicate do not. The two features that go beyond those encoders are heterogeneous graphs and attribute encoding. We measure them on domain classification of Ecore metamodels, the task of Lopez et al. [25].

Data and protocol. From ModelSet [29] we keep categories with at least ten models, drop the labels dummy and unknown, and remove near-duplicates: within a category, models whose identifier tokens have Jaccard similarity at least 0.8 are collapsed. The resulting set has 2,074 models and 48 categories. Lopez et al. report a no-duplicate split of 2,068 models and 48 categories; our file list is not theirs, so we compare methods on our split and cite their published numbers only as context. The metric is balanced accuracy, the average of the per-category recalls. Evaluation is stratified 5-fold cross-validation repeated with two seeds. Inside each training fold, 15% of each category is a validation slice. The SVM regularisation $C \in \{0.1, 1, 10\}$ and the GNN epoch are chosen on that slice; the test fold is scored once. A GNN trains for at most 30 epochs and stops after 6 epochs with no improvement of the validation balanced accuracy. The epoch with the highest validation score is the one applied to the test fold. Class weights are balanced.

Encodings. All graphs are produced by MoEGL. The name baseline is a TF-IDF vector of the element-name tokens and a linear SVM, the family of the strongest published result on this task. The MoEGL relational encoding appends tokens the bag of names cannot see, one per structural feature (owner>kind:lower-upper>type, with kind attribute, containment or reference) and one per named typed edge. The GNN comparison holds the node features fixed as the mean WordE4MDE vector of each element's name [20] and changes only the graph: a homogeneous GIN, which ignores edge types, against a heterogeneous RGCN, which uses them. A third RGCN adds the numeric structural attributes (abstract, interface, containment, bounds and the related flags) to those name vectors.

Embedding fit and the split. Table 3 does not use the dataset-wide word2vec fit of Section 5.2. The TF-IDF vectorizer is fitted on the training indices of each fold and then applied to the validation slice and the test fold. The relation tokens are read from the metamodel. The GNN node features are a lookup in the frozen WordE4MDE table, which is not estimated on this split.

Table 3. Domain classification on the near-duplicate ModelSet split (2,074 models, 48 categories). Balanced accuracy, mean $\pm$ sample standard deviation over 10 test folds. Model selection uses the validation slice only.

Encoding	Model	Balanced accuracy
Element names (TF-IDF)	Linear SVM	0.808 ± 0.024
Names plus MoEGL relation tokens	Linear SVM	0.824 ± 0.025
WordE4MDE name on each node, untyped edges	GIN	0.770 ± 0.029
Same node features, typed edges	RGCN	0.795 ± 0.036
Typed edges plus numeric attributes	RGCN	0.782 ± 0.040

On our split, Lopez et al. [25] cannot be re-scored model by model, because their duplicate list is not public. On their own no-duplicate split they report balanced accuracy 0.825 for a feed-forward network on TF-IDF names, 0.816 for an SVM on the same names, and 0.808 for a GNN on the raw graph. Our relational SVM reaches 0.824. That is level with their best published name classifier and above both their SVM and their GNN. The splits differ, so this is a comparison of published figures, not a claim that we have exceeded 0.825 on their files.

What the two features do. The relation tokens are the part of the result that the DSL adds. They raise balanced accuracy by 0.016 over names alone (standard deviation of the ten paired differences 0.017; paired $t = 2.93$, $p = 0.017$; better on 8 of the 10 folds). With the name vector of each node held fixed, the heterogeneous RGCN scores 0.795 ± 0.036 and the homogeneous GIN 0.770 ± 0.029 . The mean of the ten paired differences is 0.025 (standard deviation 0.037; paired $t = 2.14$, two-sided $p = 0.061$; the RGCN is higher on 7 of the 10 folds). The same training was repeated with the validation score stored after every

epoch, and the test scores reproduced Table 3. The selected epoch is 16.4 ± 3.8 for the GIN and 15.2 ± 4.7 for the RGCN. Training then continues until the patience limit, for 22.4 ± 3.8 and 21.2 ± 4.7 epochs. Validation balanced accuracy at the selected epoch is 0.810 ± 0.042 for the GIN and 0.822 ± 0.042 for the RGCN. On the first 15 epochs, the longest prefix reached by every fold, the mean validation score of the RGCN is higher at every epoch (0.398 against 0.165 after epoch 1). The typed graph raises the score that training reaches, at a similar number of epochs. Numeric structural attributes do not add a further gain; that RGCN scores 0.782 . The useful attribute on this task is the element name. A linear SVM on the mean WordE4MDE vector of the whole model, with no graph, scores 0.808 ± 0.024 , level with TF-IDF names (Section 5.2), so the pre-trained name vectors are available when a dataset is too small to fit word2vec. The learning claim is therefore specific: the relational graph reaches the level of the best published name classifier and is a small, statistically detectable improvement on a bag of names; the numeric flags are not what produces that score.

6.7. Scope of a Configuration

The language selects classes and features, renames them, and encodes attribute values that stay on their element. The three graph encoders of Experiments 1–3 sit inside that selection. The operations below stay outside it.

- **An attribute does not become a node.** A value such as `State.name` is stored on the State node. There is no construct that replaces the attribute with a node of its own. Making the value a node requires the value to be an object in the metamodel.
- **A reverse edge follows the opposite declared on the feature.** When a feature records an opposite, and that opposite feature exists on the target class and is not derived, the loader writes the reverse edge under the opposite's name. On the Ecore metamodel used here, `EReference.eOpposite` records no opposite, so that edge is written only where the file serializes it. Section 5.1 states the rule. There is no key that inserts any further edge.
- **An edge has no feature vector.** The edge stores the reference name in its type. Attribute encodings (`RawString`, `one-hot`, `word2vec`, `worde4mde`) apply to values on nodes. A reference such as `eType` therefore remains an edge, which is why Listing 5 encodes name and abstract and does not attach an encoding to `eType`. In the NetworkX writer the key type is the class name, so a feature that is itself named type makes that writer raise an error. The heterogeneous output keeps the class as the node type and puts the value in the feature tensor, as in Listing 4.

Table 4 places the published encoders we considered on this boundary. The three graph encoders of Experiments 1–3 are configurations of MoEGL. A graph kernel and a tree-to-sequence encoding are not configurations of MoEGL: the tool stops once the graph is written.

Table 4. Encoders MoEGL reproduces, and encodings it does not produce.

Encoder	What it produces	MoEGL
Lopez et al. [8]	Typed nodes and references, no attribute values	Reproduced: 500/500 Ecore, 500/500 RDS, 369/369 Yakindu
Rahimi et al. [22]	Class as node type and reference name as edge type, then an untyped symmetric matrix for NetGAN	The 14-line configuration produces the typed graph. The matrix step that drops direction and edge labels is theirs
Miranda et al. [23]	Transition and State nodes, source and target edges	The printed 7-node graph matches entry by entry. The 100-node file has no published graph
Clarísó and Cabot [9]	A graph kernel over models	Not produced. MoEGL writes the graph and does not compute a kernel
Burgueño et al. [10]	A containment tree read as a sequence by an LSTM	Not produced. MoEGL does not emit that sequence

7. Related Work

This section reviews some related studies that are proposed to do MDE tasks using ML models, especially those that exploit the model-to-graph encoding. MoEGL is not the first encoding of models as graphs. López et al., Rahimi et al. and Miranda et al. each wrote an encoder for one corpus, and a workshop paper proposed a DSL for homogeneous graphs [13]. The addition here is one configuration language for those encodings, together with heterogeneous graphs and attribute values. The tool writes NetworkX graphs and PyTorch Geometric heterogeneous graphs [30]. The Deep Graph Library [21] is another general graph-learning library, and MoEGL does not emit it. The classification experiment uses the Ecore models of ModelSet [29].

ML algorithms have effectively tackled various challenges in MDE [31]. For instance, model transformation rules were automatically inferred from sets of source and target models using ML techniques in [10,32]. In [33], a set of metamodels is arranged using hierarchical clustering to produce meaningful visualizations. Hierarchical clustering is also used in [11] to organize the models employing the MDEForge tool. There has also been a discussion on model encoding for clustering in [34]. Nguyen et al. [5] introduce a classification of metamodels by a tool implementing a feed-forward neural network. They propose some preprocessing tasks to transform a metamodel into a feature vector. The tasks consist of a term extractor that parses the terms in the meta-model, and then of an NLP Normalizer that normalizes the extracted terms by performing some Natural Language Processing (NLP) tasks. Burgueno et al. [10] propose a neural network architecture based on Long Short-Term Memory (LSTM) Neural Networks to perform model transformation automatically. They propose to apply a preprocessing method to represent models as trees before feeding them to an Artificial Neural Network (ANN). After transforming the model into a tree using the proposed strategy, a normalization process is applied to overcome the limitations of ANNs. Couto et al. [35] propose to classify unstructured models into metamodels using Multi-Layer Perceptrons (MLP). They translate a meta-model containing classes, attributes, and references into a set of name/value pairs in a JSON format. Then a binary vector is created as the MLP's input feature.

Some studies use model-to-graph encoding as their encoding approach. López et al. [36] address the issue of determining a generator's level of realism. They map it to a classification task where a GNN will be trained to discriminate between the two sets of models (real and synthetic). In another study, López et al. [8] propose a new architecture for the design of structurally realistic generators. This kind of generator can create new models that are essentially novel but structurally similar to the models in the dataset given a dataset

that includes multiple models considered real models. Graph kernels are suggested to be applied to MDE by the authors in [9]. Accordingly, Di Rocco et al. [37] suggest MORGAN, a graph kernel-based tool to assist in the completion of models and metamodels. The tool's idea is to suggest structural elements to finish the models that are currently being built. Specifically, MORGAN can suggest metaclasses, structural elements like attributes and references, and classes, fields, and methods inside a model class. Rahimi et al. [22] utilize generative adversarial networks (GANs) to automatically generate new structurally realistic models from a metamodel and instance model. The proposed GAN-based tool is built on the Eclipse Modeling Framework. Graph metrics are used to evaluate whether the generated artificial models match the properties of the original example model. Miranda et al. [23] introduced the concept of model views to integrate heterogeneous models in MDE, the ViewPoint Definition Language (VPDL) for specifying model views, and the EMF Views solution. They have used an ML approach introduced to support automated link inference and HGNNs as the ML technique.

Clarísó and Cabot's work [38] on user-driven diverse scenario exploration in model finders presents a domain-specific language (DSL) aimed at controlling how Alloy specifications are encoded into graphs for clustering purposes. Their DSL allows users to define scenarios by selecting and filtering model elements and attributes, providing a flexible approach to model exploration and analysis. This method, while targeting Alloy specifications, demonstrates the broader applicability and influence of DSLs in model-driven engineering. The concepts of selecting and filtering elements in their DSL are closely aligned with our approach, emphasizing the importance of user control and customization in model encoding processes. This highlights the versatility of DSLs in enhancing the usability and precision of model-driven tasks.

Each of those encoders is written for one task. MoEGL expresses the three graph encoders of Section 6 by a configuration of 14 to 29 lines. The learning task measured here is domain classification: relation tokens raise balanced accuracy from 0.808 to 0.824.

Five publications from 2025 and 2026 still depend on a representation of the model. López et al. [39] separate encoding from training in a benchmark harness, and record that a GNN requires the models in a graph format. López et al. [40] extend the WordE4MDE vectors with subword units, a BERT model trained on modelling text, and additional forum text. Section 6.6 uses the skip-gram vectors of the 2023 paper [20]. Khalilipour et al. [41] turn each Ecore metamodel into a semantic attributed graph and classify it with a GNN into 24 categories, using BERT or TF-IDF features. They report an accuracy of about 96% for the BERT model on that retrieval task. The category set, the metric and the features differ from Section 6.6. Chen et al. [42] generate a graph model from text so that the result satisfies a metamodel. Ali [43] encodes Ecore, OntoUML and ArchiMate models as knowledge graphs and trains a GNN together with a language model. These works consume a graph or a name vector. The configuration that chooses the graph is the subject of this article.

7.1. Domain-Specific Transformation Languages (DSTLs)

Csertán et al. [44] presented VIATRA, a language that transforms a model into another model for verification. Varró is a co-author of that paper. MoEGL does not transform a model into another model. It writes a graph for a learner.

8. Evaluation of Research Contributions and Questions

Each question is answered in the experiment that measures it. The figures are stated there, and they are not copied here.

- **RQ1 (Fidelity).** Experiment 1 (Section 6) compares the MoEGL graphs with the published graphs.

- **RQ2 (Cost).** Section 6.5 and Table 2 report specification size, encoding time and peak memory.
- **RQ3 (Effect).** Section 6.6 and Table 3 report the classification result, including the homogeneous and heterogeneous models.

Together these three results are the evaluation of the contribution. The boundary of the language is Section 6.7.

9. Conclusion and Future Work

In this paper, we introduced MoEGL, a domain-specific language designed to encode models into graphs compatible with graph neural networks. One configuration reproduces the published graphs of López et al. (500 Ecore, 500 RDS, 369 Yakindu), the per-task specification is 14 to 29 lines of YAML, and relation tokens raise balanced accuracy from 0.808 to 0.824 on domain classification. Future work will explore further enhancements to MoEGL and its application to other MDE tasks.

The work sits next to the encoders of López et al., Rahimi et al. and Miranda et al., and next to the 2021 workshop DSL. What this article adds, and measures, is one shared configuration for those encodings, heterogeneous graphs, attribute-value encodings, and the classification result in Section 6.6.

Looking ahead, we aim to apply MoEGL to further learning tasks on models. Some potential avenues of future work include considering alternate encodings for attribute values within MoEGL, allowing users to filter model elements according to Object Constraint Language (OCL) expressions defined in the MoEGL configuration, and more fully realizing the proposed heterogeneous graph encoding approach. A goal is to semantically encode the element names in models to facilitate various MDE activities. Another possible extension of this research going forward involves designing heuristics to automatically determine well-suited encodings based on analyzing a metamodel's structure and characterizing the ML objective. Moving ahead, we plan to extensively research the applicability of encoding models as heterogeneous graphs for a wide range of MDE use cases. We also intend to support other popular GNNs and HGNNs like the new implementation of heterogeneous graphs in tensorflow [45].

Author Contributions: Conceptualization, Z.R. and S.K.-R.; methodology, Z.R.; software, Z.R.; validation, Z.R., S.K.-R. and M.T.; formal analysis, Z.R.; investigation, Z.R.; resources, Z.R.; data curation, Z.R.; writing—original draft preparation, Z.R.; writing—review and editing, S.K.-R. and M.T.; visualization, Z.R.; supervision, S.K.-R. and M.T.; project administration, S.K.-R.; funding acquisition, S.K.-R. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The source code of MoEGL, the configurations of the three experiments, the measurement scripts (fidelity check, timing, lines-of-code count), the driver used to run the bespoke encoder of Experiment 1 and the raw results are openly available in the MoEGL repository at <https://github.com/Z-Rajaei/MoEGL>. The running-example metamodel, instance and the four YAML listings are in `examples/running_fsm`, and `examples/running_fsm/check_listings.py` is the script that executed them. The 500 Ecore models and the published graphs of Experiment 1 are part of the replication package of Lopez et al. (TCRMG-GNN, <https://github.com/Antolin1/TCRMG-GNN>).

Acknowledgments: During the preparation of this manuscript, the authors used generative AI tools (e.g., ChatGPT and GitHub Copilot) for language polishing and code assistance. The authors have reviewed and edited the output and take full responsibility for the content of this publication.

Conflicts of Interest: The authors declare no conflicts of interest. The funders had no role in the design of the study; in the collection, analyses, or interpretation of data; in the writing of the manuscript; or in the decision to publish the results.

Abbreviations

The following abbreviations are used in this manuscript:

MDE	Model-Driven Engineering
DSL	Domain-Specific Language
GNN	Graph Neural Network
HGNN	Heterogeneous Graph Neural Network
ML	Machine Learning
AI	Artificial Intelligence
EMF	Eclipse Modeling Framework
UML	Unified Modeling Language
XMI	XML Metadata Interchange
OCL	Object Constraint Language
GAN	Generative Adversarial Network
LSTM	Long Short-Term Memory
MLP	Multi-Layer Perceptron
NLP	Natural Language Processing
MoEGL	Model Encoding for Graph Learning
HPyG	Heterogeneous PyTorch Geometric
GCN	Graph Convolutional Network
GAT	Graph Attention Network
LoC	Lines of Code
DOI	Digital Object Identifier
CSV	Comma-Separated Values
FSM	Finite State Machine
URI	Uniform Resource Identifier
URL	Uniform Resource Locator
YAML	YAML Ain't Markup Language

References

1. Brambilla, M.; Cabot, J.; Wimmer, M. *Model-driven software engineering in practice*; Morgan & Claypool Publishers, 2017.
2. Mellor, S.J.; Clark, A.N.; Futagami, T. Guest editors' introduction: Model-driven development. *IEEE Software* **2003**, *20*, 14–18.
3. Whittle, J.; Hutchinson, J.; Rouncefield, M. The state of practice in model-driven engineering. *IEEE software* **2013**, *31*, 79–85.
4. Panat, S.; Kumar, R. Introduction to Artificial Intelligence & ML. In *Introduction to Artificial Intelligence & ML*; Springer, 2023; pp. 127–146.
5. Nguyen, P.T.; Rocco, J.D.; Ruscio, D.D.; Pierantonio, A.; Iovino, L. Automated classification of metamodel repositories: a machine learning approach. In Proceedings of the Proceedings of the IEEE International Conference on Software Engineering (ICSE). IEEE, 2019, pp. 272–282.
6. Osman, M.H.; Ho-Quang, T.; Chaudron, M. An automated approach for classifying reverse-engineered and forward-engineered UML class diagrams. In Proceedings of the 2018 44th Euromicro Conference on Software Engineering and Advanced Applications (SEAA). IEEE, 2018, pp. 396–399.
7. Barriga, A.; Rutle, A.; Heldal, R. Personalized and automatic model repairing using reinforcement learning. In Proceedings of the 2019 ACM/IEEE 22nd International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C). IEEE, 2019, pp. 175–181.
8. López, J.A.H.; Cuadrado, J.S. Generating structurally realistic models with deep autoregressive networks. *IEEE Transactions on Software Engineering* **2022**, *49*, 2661–2676.

9. Clarisó, R.; Cabot, J. Applying graph kernels to model-driven engineering problems. In Proceedings of the Proceedings of the 1st International Workshop on Machine Learning and Software Engineering in Symbiosis. ACM, 2018, pp. 1–5.
10. Burgueño, L.; Cabot, J.; Gérard, S. An LSTM-based neural network architecture for model transformations. In Proceedings of the 2019 ACM/IEEE 22nd International Conference on Model Driven Engineering Languages and Systems (MODELS). IEEE, 2019, pp. 294–299.
11. Basciani, F.; Di Rocco, J.; Di Ruscio, D.; Iovino, L.; Pierantonio, A. Automated clustering of metamodel repositories. In Proceedings of the Advanced Information Systems Engineering: 28th International Conference, CAiSE 2016, Ljubljana, Slovenia, June 13–17, 2016. Proceedings 28. Springer, 2016, pp. 342–358.
12. Ali, S.J.; Gavric, A.; Proper, H.; Bork, D. Encoding Conceptual Models for Machine Learning: A Systematic Review. In Proceedings of the 2023 ACM/IEEE International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C). IEEE, 2023, pp. 562–570.
13. Rajaei, Z.; Kolahdouz-Rahimi, S.; Tisi, M.; Jouault, F. A DSL for Encoding Models for Graph-Learning Processes. In Proceedings of the 20th International Workshop on OCL and Textual Modeling. CEUR-WS.org, 2021.
14. Gilmer, J.; Schoenholz, S.S.; Riley, P.F.; Vinyals, O.; Dahl, G.E. Neural message passing for quantum chemistry. In Proceedings of the International conference on machine learning. PMLR, 2017, pp. 1263–1272.
15. Scarselli, F.; Gori, M.; Tsoi, A.C.; Hagenbuchner, M.; Monfardini, G. The graph neural network model. *IEEE transactions on neural networks* **2008**, *20*, 61–80.
16. Xu, K.; Hu, W.; Leskovec, J.; Jegelka, S. How powerful are graph neural networks? *arXiv preprint arXiv:1810.00826* **2018**.
17. Sun, Y.; Han, J. *Mining heterogeneous information networks: principles and methodologies*; Morgan & Claypool Publishers, 2012.
18. Aranega, V. Pyecore: A Python (Ic) Implementation of the Eclipse Modeling Framework. *modeling-languages. com* **2017**.
19. Ben-Kiki, O.; Evans, C.; Ingerson, B. Yaml ain't markup language (yaml™) version 1.1. *Working Draft 2008-05* **2009**, *11*.
20. López, J.A.H.; Durá, C.; Cuadrado, J.S. Word embeddings for model-driven engineering. In Proceedings of the 2023 ACM/IEEE 26th International Conference on Model Driven Engineering Languages and Systems (MODELS). IEEE, 2023, pp. 151–161.
21. Wang, M.; Yu, L.; Zheng, D.; Gan, Q.; Gai, Y.; Ye, Z.; Li, M.; Zhou, J.; Huang, Q.; Ma, C.; et al. Deep Graph Library: Towards Efficient and Scalable Deep Learning on Graphs. **2019**.
22. Rahimi, A.; Tisi, M.; Rahimi, S.K.; Berardinelli, L. Towards Generating Structurally Realistic Models by Generative Adversarial Networks. In Proceedings of the 2023 ACM/IEEE International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C). IEEE, 2023, pp. 597–604.
23. Miranda, J.P.; Bruneliere, H.; Tisi, M.; Sunyá, G. Towards the Integration Support for Machine Learning of Inter-Model Relations in Model Views. In Proceedings of the The 39th ACM/SIGAPP Symposium on Applied Computing (SAC'24). ACM, 2024.
24. Morris, C.; Kriege, N.M.; Bause, F.; Kersting, K.; Mutzel, P.; Neumann, M. TUDataset: A collection of benchmark datasets for learning with graphs. *arXiv preprint arXiv:2007.08663* **2020**.
25. López, J.A.H.; Rubei, R.; Cuadrado, J.S.; Di Ruscio, D. Machine learning methods for model classification: A comparative study. In Proceedings of the Proceedings of the 25th International Conference on Model Driven Engineering Languages and Systems. ACM, 2022, pp. 165–175.
26. Tahmasivand, A.; Zahran, N.; Al-Sayouri, S.; Fouda, M.; Khasawneh, K.N. LM-Fix: Lightweight Bit-Flip Detection and Rapid Recovery Framework for Language Models. In Proceedings of the 2025 IEEE 43rd International Conference on Computer Design (ICCD). IEEE, 2025, pp. 432–440.
27. Zahran, N.; Tahmasivand, A.; Alouani, I.; Khasawneh, K.; Fouda, M.E. On Jailbreaking Quantized Language Models Through Fault Injection Attacks. In Proceedings of the Proceedings of the Great Lakes Symposium on VLSI 2025, 2025, pp. 554–561.
28. Omid, B.; Tahmasivand, A.; Alsayouri, H.; Al-Sayouri, S.; Fang, C.; Alouani, I.; Khasawneh, K.N. Faults That Fortify: CNN Adversarial Robustness via GPU Undervolting, 2026. *arXiv:2608.20572*.
29. López, J.A.H.; Cánovas Izquierdo, J.L.; Cuadrado, J.S. ModelSet: a dataset for machine learning in model-driven engineering. *Software and Systems Modeling* **2022**, *21*, 967–986. <https://doi.org/10.1007/s10270-021-00929-3>.
30. Fey, M.; Lenssen, J.E. Fast graph representation learning with PyTorch Geometric. *arXiv preprint arXiv:1903.02428* **2019**.
31. Babur, Ö.; Chaudron, M.R.; Cleophas, L.; Di Ruscio, D.; Kolovos, D. AMMoRe 2018: First international workshop on analytics and mining of model repositories. In Proceedings of the Proceedings of MODELS 2018 Workshops: ModComp, MRT, OCL, FlexMDE, EXE, COMMitMDE, MDETools, GEMOC, MORSE, MDEIoT, MDEbug, MoDeVva, ME, MULTI, HuFaMo, AMMoRe, PAINS. CEUR Workshop Proceedings, 2018, Vol. 2245, pp. 778–779.
32. Dolques, X.; Huchard, M.; Nebut, C.; Reitz, P. Learning transformation rules from transformation examples: An approach based on relational concept analysis. In Proceedings of the 2010 14th IEEE International Enterprise Distributed Object Computing Conference Workshops. IEEE, 2010, pp. 27–32.
33. Babur, Ö.; Cleophas, L.; van den Brand, M. Hierarchical clustering of metamodels for comparative analysis and visualization. In Proceedings of the European conference on modelling foundations and applications. Springer, 2016, pp. 3–18.

34. Babur, Ö.; Cleophas, L. Using n-grams for the automated clustering of structural models. In Proceedings of the International Conference on Current Trends in Theory and Practice of Informatics. Springer, 2017, pp. 510–524. 1070
35. Couto, W.O.; Morais, E.C.; Del Fabro, M.D. Classifying Unstructured Models into Metamodels using Multi Layer Perceptrons. In Proceedings of the MODELSWARD. SCITEPRESS, 2020, pp. 271–278. 1071
36. López, J.A.H.; Cuadrado, J.S. Towards the characterization of realistic model generators using graph neural networks. In Proceedings of the 2021 ACM/IEEE 24th International Conference on Model Driven Engineering Languages and Systems (MODELS). IEEE, 2021, pp. 58–69. 1072
37. Di Rocco, J.; Di Sipio, C.; Di Ruscio, D.; Nguyen, P.T. A GNN-based Recommender System to Assist the Specification of Metamodels and Models. In Proceedings of the 2021 ACM/IEEE 24th International Conference on Model Driven Engineering Languages and Systems (MODELS). IEEE, 2021. 1073
38. Clarisó, R.; Cabot, J. User-driven diverse scenario exploration in model finders. *Science of Computer Programming* **2022**, *215*, 102745. 1074
39. López, J.A.H.; Cuadrado, J.S.; Rubel, R.; Di Ruscio, D. ModelXGlue: a benchmarking framework for ML tools in MDE. *Software and Systems Modeling* **2025**, *24*, 1035–1058. <https://doi.org/10.1007/s10270-024-01183-z>. 1075
40. López, J.A.H.; Durá, C.; Cuadrado, J.S. Experimenting with modeling-specific word embeddings. *Software and Systems Modeling* **2025**, *24*, 1647–1669. <https://doi.org/10.1007/s10270-024-01250-5>. 1076
41. Khalilipour, A.; Bozyiğit, F.; Challenger, M. Context-aware semantic model retrieval using graph neural networks. In Proceedings of the 2025 ACM/IEEE 28th International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C). IEEE, 2025, pp. 1–10. <https://doi.org/10.1109/MODELS-C68889.2025.11403961>. 1077
42. Chen, B.; Wei, O.; Zheng, B.; Mussbacher, G. Accurate and consistent graph model generation from text with large language models. In Proceedings of the 2025 ACM/IEEE 28th International Conference on Model Driven Engineering Languages and Systems (MODELS). IEEE, 2025, pp. 130–141. <https://doi.org/10.1109/MODELS67397.2025.00018>. 1078
43. Ali, S.J. From Symbols to Signals: Representation Learning in Conceptual Modeling. PhD thesis, Technische Universität Wien, 2026. <https://doi.org/10.34726/hss.2026.106743>. 1079
44. Csértán, G.; Huszerl, G.; Majzik, I.; Pap, Z.; Pataricza, A.; Varró, D. VIATRA-visual automated transformations for formal verification and validation of UML models. In Proceedings of the Proceedings 17th IEEE International Conference on Automated Software Engineering. IEEE, 2002, pp. 267–270. 1080
45. Graph neural networks in TensorFlow. <https://blog.tensorflow.org/2024/02/graph-neural-networks-in-tensorflow.html>. Accessed: 2024-03-20. 1081

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content. 1082